The 20 % of Python & PyTorch you need

Module 2 · Zero to LLM

Six ideas, that's all

Python	PyTorch
lists + comprehensions	tensors
dicts (vocabularies!)	shapes
functions & classes	autograd + `nn.Module`

Tensors = spreadsheets with more dimensions

- scalar `5` → vector `[5, 2]` → matrix (grid) → 3-D stack
- LLM standard: **batch × tokens × features** e.g. `(8, 16, 768)`
- Habit: `print(x.shape)` — every time

Matrix multiply

`(2×3) @ (3×4) → (2×4)` — inner sizes must match

"Every row asks every column a question"

Autograd in one picture

`loss.backward()` → each parameter gets `.grad` = "which way is downhill"

`w -= lr * w.grad` then **zero the grad**

Gotchas

1. Keyword names: `nn.Embedding(num_embeddings, embedding_dim)`, `nn.LayerNorm(768)`
2. Forgot `zero_grad()` → gradients pile up
3. `long` token IDs vs `float` weights
4. Model and data on different devices
5. GPT-2 **and BERT** learn position embeddings; fixed sinusoids = 2017 Transformer; modern open LLMs → RoPE

Next: M03 — Text → token IDs